



**B.Sc. (General/Special) Degree
Programme
Faculty of Applied Sciences
University of Sri Jayewardenepura**

ICT 1022 2.0 Computer System Architecture

Presented By:

Surani Tissera(PhD)

Department of Computer Science

Number System



This is a property of Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.

Number System: The Basics

- We are accustomed to the so-called decimal number system.
 - Ten digits :: 0,1,2,3,4,5,6,7,8,9
 - Every digit position has a weight which is a power of 10.
 - Base or radix is 10.

- Examples:

$$234 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

$$250.67 = 2 \times 10^2 + 5 \times 10^1 + 0 \times 10^0 + 6 \times 10^{-1} + 7 \times 10^{-2}$$



Binary Number System

- Two digits: 0 and 1.
 - Every digit position has a weight that is a power of 2.
 - Base or radix is 2.

- Examples:

$$110 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$101.01 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$



Binary to Decimal Conversion

- Each digit position of a binary number has a weight.
 - Some power of 2.

A binary number:

$$B = b_{n-1} b_{n-2} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-m}$$

where b^i are the binary digits.

Corresponding value in decimal: $n-1$

$$D = \sum_{i=-m}^n b_i 2^i$$



Some Examples

1. 101011

2. .0101

3. 101.11



Decimal to Binary Conversion

- Consider the integer and fractional parts separately.
- For the integer part:
 - Repeatedly divide the given number by 2, and go on accumulating the remainders, until the number becomes zero.
 - Arrange the remainders in reverse order.
- For the fractional part:
 - Repeatedly multiply the given fraction by 2.
- Accumulate the integer part (0 or 1).
- If the integer part is 1, chop it off.
 - Arrange the integer parts in the order they are obtained.



Decimal to Binary Conversion

2	239	
2	119	---
2	59	---
2	29	---
2	14	---
2	7	---
2	3	---
2	1	---
2	0	---

$(239)_{10} = (11101111)_2$

2	64	
2	32	---
2	16	---
2	8	---
2	4	---
2	2	---
2	1	---
2	0	---

$(64)_{10} = (1000000)_2$

.634	x 2	=	1.268
.268	x 2	=	0.536
.536	x 2	=	1.072
.072	x 2	=	0.144
.144	x 2	=	0.288
:			
$(.634)_{10} = (.10100.....)_2$			

37.0625
$(37)_{10} = (100101)_2$
$(.0625)_{10} = (.0001)_2$
$\therefore (37.0625)_{10} =$
$(100101.0001)_2$



Hexadecimal Number System

A compact way to represent binary numbers.

- Group of four binary digits are represented by a hexadecimal digit.
- Hexadecimal digits are 0 to 9, A to F.

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111



Binary to Hexadecimal Conversion

- For the integer part:
 - Scan the binary number from *right to left*.
 - Translate each group of four bits into the corresponding hexadecimal digit.
 - Add *leading* zeros if necessary.
- For the fractional part:
 - Scan the binary number from *left to right*.
 - Translate each group of four bits into the corresponding hexadecimal digit.
 - Add *trailing* zeros if necessary.



Examples

1. $(1011\ 0100\ 0011)_2$

2. $(10\ 1010\ 0001)_2$

3. $(.1000\ 010)_2$

4. $(101\ .\ 0101\ 111)_2$



Hexadecimal to Binary Conversion

- Translate every hexadecimal digit into its 4-bit binary equivalent.
- Examples:

$$(3A5)_{16} =$$

$$(12.3D)_{16} =$$

$$(1.8)_{16} =$$



How are Hexadecimal Numbers Written?

- Using the suffix “H” or using the prefix “0x”.
- Examples:
 - `ADDI R1,2AH` // Add the hex number 2A to register R1
 - `0x2AB4` // The 16-bit number 0010 1010 1011 0100
 - `0xFFFFFFFF` // The 32-bit number for the all-1 string



Unsigned Binary Numbers

- An n-bit binary number can have 2^n distinct combinations.
 - For example,
for $n=3$, the 8 distinct combinations are:
000, 001, 010, 011, 100, 101, 110, 111 (0 to $2^3-1 = 7$ in decimal).

Number of bits (n)	Range of Numbers
8	0 to 2^8-1 (255)
16	0 to $2^{16}-1$ (65535)
32	0 to $2^{32}-1$ (4294967295)
64	0 to $2^{64}-1$



Unsigned Binary Numbers

- An n-bit binary integer: $b_{n-1}b_{n-2} \dots b_2b_1b_0$

- Equivalent unsigned decimal value:

$$D = b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_22^2 + b_12^1 + b_02^0$$

- Each digit position has a weight that is some power of 2.



Signed Integer Representation

- Many of the numerical data items that are used in a program are signed (positive or negative).
 - Question:: How to represent sign?
- Three possible approaches:
 - a) Sign-magnitude representation
 - b) One's complement representation
 - c) Two's complement representation



(a) Sign-magnitude Representation

- For an n-bit number representation:
 - The most significant bit (MSB) indicates sign (0: positive, 1: negative).
 - The remaining (n-1) bits represent the magnitude of the number.
- Range of numbers: $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$



- A problem: Two different representations for zero. +0: 0 00..000 and -0: 1 00..000



(b) One's Complement Representation

- Basic idea:
 - Positive numbers are represented exactly as in sign-magnitude form.
 - Negative numbers are represented in 1's complement form.
- How to compute the 1's complement of a number?
 - Complement every bit of the number (1→0 and 0→1).
 - MSB will indicate the sign of the number (0: positive, 1: negative).



Example for n=4

Decimal	1's complement	Decimal	1's complement
+0	0000	-7	1000
+1	0001	-6	1001
+2	0010	-5	1010
+3	0011	-4	1011
+4	0100	-3	1100
+5	0101	-2	1101
+6	0110	-1	1110
+7	0111	-0	1111

To find the representation of,
say,

-4, first note that

+4 = 0100

-4 = 1's complement of 0100
= 1011



- Range of numbers that can be represented in 1's complement:

Maximum + $(2^{n-1} - 1)$

Minimum - $(2^{n-1} - 1)$

- A problem:

Two different representations of zero.

+0 -> 0 000....0

-0 -> 1 111....1

- Advantage of 1's complement representation:
 - Subtraction can be done using addition.
 - Leads to substantial saving in circuitry.



(c) Two's Complement Representation

- Basic idea:
 - Positive numbers are represented exactly as in sign-magnitude form.
 - Negative numbers are represented in 2's complement form.
- How to compute the 2's complement of a number?
 - Complement every bit of the number ($1 \rightarrow 0$ and $0 \rightarrow 1$), and then add one to the resulting number.
 - MSB will indicate the sign of the number (0: positive, 1: negative).



Example for n=4

Decimal	2's complement	Decimal	2's complement
+0	0000	-8	1000
+1	0001	-7	1001
+2	0010	-6	1010
+3	0011	-5	1011
+4	0100	-4	1100
+5	0101	-3	1101
+6	0110	-2	1110
+7	0111	-1	1111

To find the representation of,
say,

-4, first note that

$$+4 = 0100$$

$$\begin{aligned} -4 &= 2\text{'s complement of } 0100 \\ &= 1011 + 1 \\ &= 1100 \end{aligned}$$



Example for $n=4$

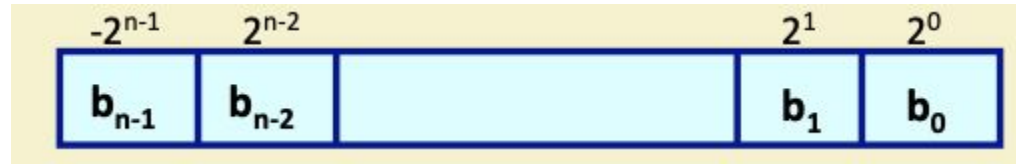
- Range of numbers that can be represented in 2's complement:
Maximum $+ (2^{n-1} - 1)$
Minimum $- 2^{n-1}$
- Advantage of 2's complement representation:
 - Unique representation of zero.
 - Subtraction can be done using addition.
 - Leads to substantial saving in circuitry.
- Almost all computers today use 2's complement representation for storing negative numbers.



Example for n=4

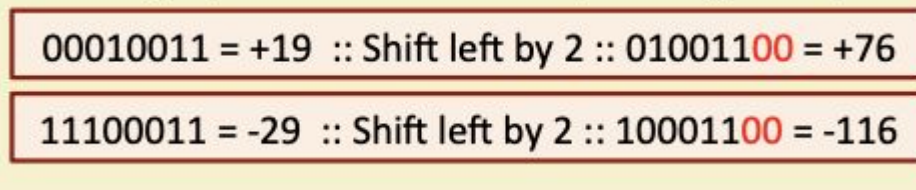
Some other features of 2's complement representation

a) Weighted number representation, with the MSB having weight -2^{n-1} .



$$D = -b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_22^2 + b_12^1 + b_02^0$$

b) Shift left by k positions with zero padding multiplies the number by 2^k .



Example for n=4

c) Shift right by k positions with sign bit padding divides the number by 2^k .

00010110 = +22 :: Shift right by 2 :: 00000101 = +5

11100100 = -28 :: Shift right by 2 :: 11111001 = -7

d) The sign bit can be copied as many times as required in the beginning to extend the size of the number (called sign extension).

X = 00101111 (8-bit number, value = +47)

Sign extend to 32 bits:

00000000 00000000 00000000 00101111

X = 10100011 (8-bit number, value = -93)

Sign extend to 32 bits:

11111111 11111111 11111111 10100011



Thank
you!

dreamstime

